

Experiment No.10

1. **Title:** Chat Application using Multithreading.

2. **Problem Statement:**

Develop a multithreaded Java application that simulates a real-time application such as a basic chat system. Each user is represented by a separate thread that can send and receive messages concurrently. The application demonstrates thread management using `isAlive()`, `join()`, thread priorities, and controlled thread suspension, resumption, and stopping.

3. **Objectives:**

- Implement **multiple threads** to represent different users.
- Demonstrate **concurrent message sending and receiving**.
- Use `isAlive()` and `join()` for **thread monitoring and synchronization**.
- Implement **thread priorities** for handling important messages.
- Demonstrate thread **suspension, resumption, and stopping**.

4. **Outcomes:**

After completing this problem, students will be able to:

- Create and manage multiple threads in Java.
- Explain **concurrent execution** in a real-time application.
- Use `isAlive()` and `join()` effectively.
- Apply **thread priorities** to manage task importance.
- Control the execution and lifecycle of threads.

5. **Software/Hardware/Other Requirements:**

- Any CPU with Pentium Processor / similar, 256 MB RAM/ more, 1GB HDD / more.
- Operating System – ubuntu/Fedora 64bit OS
- Software: JDK 17 or later, Code Editor

6. **Theory:**

A. `isAlive()` and `join()` Methods

- Even though threads are independent most of the time, there might some instances at which we want a certain thread to wait until all other threads complete their execution. In such situations we can use `isAlive()` and `join()` methods.
- The `isAlive()` method can be used to find whether the thread is still running or not. If the thread is still running, it will return true. Otherwise, it will return false.
- The `join()` method makes the thread to wait until the invoking thread terminates.
- Syntax:

`final boolean isAlive()`

`final void join() throws InterruptedException`

`final void join(long milliseconds) throws InterruptedException`

```
class MyThread implements Runnable {
    Thread t;
    MyThread() {
        t = new Thread(this);
        System.out.println("Child thread: " + t);
        t.start();
    }
    public void run() {
        try {
            for(int i = 1; i <= 3; i++) {
                System.out.println(t.getName() + ": " + i);
                Thread.sleep(500);
            }
        }
    }
}
```

```
    }
    catch(InterruptedException e) {
        System.out.println("Child thread is interrupted!");
    }
    System.out.println("Child thread is terminated");
}
}
public class Driver {
    public static void main(String[] args) {
        MyThread my = new MyThread();
        try {
            System.out.println("Child thread is running: " + my.t.isAlive());
            System.out.println("Main thread is waiting...");
            my.t.join();
        }
        catch(InterruptedException e) {
            System.out.println("Main thread is interrupted!");
        }
        System.out.println("Child thread is running: " + my.t.isAlive());
        System.out.println("Main thread terminated");
    }
}
```

Output:

```
Child thread: Thread[#20,Thread-0,5,main]
Child thread is running: true
Main thread is waiting...
Thread-0: 1
Thread-0: 2
Thread-0: 3
Child thread is terminated
Child thread is running: false
Main thread terminated
```

In the above program, main thread is waiting for the child thread to complete its execution and then it terminates.

B. Priority of a Thread in Java

- Every Java thread has a priority that helps the operating system determine the order in which threads are scheduled. You can get and set the priority of a Thread.
- Thread class provides methods and constants for working with the priorities of a Thread.
- Threads with higher priority are more important to a program and should be allocated processor time before lower-priority threads.
- However, thread priorities cannot guarantee the order in which threads execute and are very much platform dependent.
- **Built-in Property Constants of Thread Class**
 - Java thread priorities are in the range between MIN_PRIORITY (a constant of 1) and MAX_PRIORITY (a constant of 10). By default, every thread is given priority NORM_PRIORITY (a constant of 5).
 - **MIN_PRIORITY**: Specifies the minimum priority that a thread can have.
 - **NORM_PRIORITY**: Specifies the default priority that a thread is assigned.
 - **MAX_PRIORITY**: Specifies the maximum priority that a thread can have.
- **Thread Priority Setter and Getter Methods**
 - **Thread.getPriority() Method**: This method is used to get the priority of a thread.
 - **Thread.setPriority() Method**: This method is used to set the priority of a thread, it accepts the priority value and updates an existing priority with the given priority.

a) **Thread.getPriority() Method:**

- The java.lang.Thread.getPriority() method returns the priority of the given thread.
- **Syntax:** public final int getPriority()

b) **Thread.setPriority() Method:**

- The java.lang.Thread.setPriority() method updates or assign the priority of the thread to newPriority.
- The method throws IllegalArgumentException if the value newPriority goes out of the range, which is 1 (minimum) to 10 (maximum).
- **Syntax:** public final void setPriority(int newPriority)

```
public class ThreadPriority extends Thread {
    public void run() {
        System.out.println("Inside the run() method");
    }
    public static void main(String args[]) {
        ThreadPriority tv1 = new ThreadPriority();
        ThreadPriority tv2 = new ThreadPriority();
        ThreadPriority tv3 = new ThreadPriority();
        System.out.println("Priority of the thread tv1 is : " + tv1.getPriority());
        System.out.println("Priority of the thread tv2 is : " + tv2.getPriority());
        System.out.println("Priority of the thread tv2 is : " + tv2.getPriority());
        tv1.setPriority(6);
        tv2.setPriority(3);
        tv3.setPriority(9);
        System.out.println("Priority of the thread tv1 is : " + tv1.getPriority());
        System.out.println("Priority of the thread tv2 is : " + tv2.getPriority());
        System.out.println("Priority of the thread tv3 is : " + tv3.getPriority());
        System.out.println("Currently Executing the Thread : " + Thread.currentThread().getName());
        System.out.println("Priority of the main thread is : " + Thread.currentThread().getPriority());
        Thread.currentThread().setPriority(10);
        System.out.println("Priority of the main thread is : " + Thread.currentThread().getPriority());
    }
}
```

Output:

```
Priority of the thread tv1 is : 5
Priority of the thread tv2 is : 5
Priority of the thread tv2 is : 5
Priority of the thread tv1 is : 6
Priority of the thread tv2 is : 3
Priority of the thread tv3 is : 9
Currently Executing The Thread : main
Priority of the main thread is : 5
Priority of the main thread is : 10
```

C. How Thread Priority Affects CPU Scheduling in Java

- Thread priority is a hint to the JVM thread scheduler about which thread should get CPU time first when multiple threads are ready to run. However, it does not guarantee execution order because the final decision depends on the JVM implementation and the operating system scheduler.
- **CPU Scheduling Example:** Assume three runnable threads:

Thread	Priority
T1	2
T2	5
T3	8

- When the CPU becomes available, the scheduler is more likely to select T₃ first because it has the highest priority. After T₃ blocks, sleeps, yields, or completes, the scheduler may choose T₂ and then T₁.

- **What Happens Internally?**
 1. All runnable threads enter the scheduler's queue.
 2. The scheduler examines thread priorities.
 3. Threads with higher priority are generally favored for CPU allocation.
 4. If a higher-priority thread becomes runnable while a lower-priority thread is executing, the scheduler may preempt the lower-priority thread and run the higher-priority one.

<pre> class MyThread extends Thread { public MyThread(String name) { super(name); } public void run() { for(int i = 1; i <= 5; i++) { System.out.println(getName() + " Priority: " + getPriority() + " Count: " + i); } } } public class ThreadPriorityDemo { public static void main(String[] args) { MyThread low = new MyThread("LOW"); MyThread normal = new MyThread("NORMAL"); MyThread high = new MyThread("HIGH"); low.setPriority(Thread.MIN_PRIORITY); // 1 normal.setPriority(Thread.NORM_PRIORITY); // 5 high.setPriority(Thread.MAX_PRIORITY); // 10 low.start(); normal.start(); high.start(); } } </pre>	Output: HIGH Priority: 10 Count: 5 LOW Priority: 1 Count: 1 LOW Priority: 1 Count: 2 LOW Priority: 1 Count: 3 NORMAL Priority: 5 Count: 3 NORMAL Priority: 5 Count: 4 NORMAL Priority: 5 Count: 5 LOW Priority: 1 Count: 4 LOW Priority: 1 Count: 5
---	---

D. Thread Execution

- A scheduling method known as fixed-priority pre-emptive scheduling is supported by the JVM.
- Every Java thread has a priority, and the JVM gives the highest priority thread the most attention first.
- A Thread inherits its default priority when it is created. The JVM chooses and executes the Runnable thread with the highest priority when several threads are prepared to run.
- The lower priority threads will run in the event that this thread is terminated or becomes unrunnable. The JVM will execute threads in FIFO order if they have the same priority.
- **There are two situations that may result in the execution of a separate thread:**
 - The ability to execute a thread with a greater priority than the one now running.
 - The current thread either yields (briefly pauses to allow other threads) or leaves the runnable state.
 - In general, the thread with the highest priority is active at all times. But occasionally, the thread scheduler may decide to execute low-priority threads in order to prevent starvation.

E. Thread Synchronization

- Thread Synchronization is a mechanism that ensures only one thread can access a shared resource at a time, preventing data inconsistency and race conditions. The synchronized keyword so that only one thread can execute it at a time for a specific object instance.
- A **synchronized method** in Java is a method that is modified with the synchronized keyword.
- It ensures that only one thread can access the method at a time for a particular object instance.
- If another thread tries to call any synchronized method on the same object, it will be blocked until the first thread finishes execution.
- **Use of Synchronized Method :-**
 - **Thread Safety:** Prevents race conditions (when multiple threads modify shared data at the same time). Ensures atomicity (a complete operation is executed without interruption).

- **Mutual Exclusion (Mutex):** Provides exclusive access to shared resources (e.g., shared variables, files, database connections).
- **Consistency of Data:** Prevents data inconsistency caused by concurrent modifications.
- **Easy to Use:** Simpler to implement than using synchronized blocks manually, since the entire method becomes synchronized.

- **Syntax:-**

```
synchronized returnType methodName(parameters)
```

```
{
```

```
    // method body
```

```
}
```

```
class BookMovieSeat {
    int total_seats=10;
    synchronized void bookSeat(int seats, String name) {
        if(total_seats>=seats) {
            System.out.println(name+" booked "+seats+" seats successfully");
            total_seats=total_seats-seats;
            System.out.println("Total seats left : "+total_seats);
        }
        else {
            System.err.println(name+" sorry...!! seats not left");
            System.err.println("Total seats left : "+total_seats);
        }
    }
}

class MyThread extends Thread {
    BookMovieSeat bts;
    int seats;
    public MyThread(BookMovieSeat bts, int seats) {
        this.bts=bts;
        this.seats=seats;
    }
    public void run() {
        bts.bookSeat(seats, Thread.currentThread().getName());
    }
}

public class MovieBookingApp {
    public static void main(String[] args) {
        BookMovieSeat bts=new BookMovieSeat();
        MyThread deepak=new MyThread(bts, 6);
        deepak.setName("Deepak");
        deepak.start();
        MyThread amit=new MyThread(bts, 5);
        amit.setName("Amit");
        amit.start();
    }
}
```

Output

Deepak booked 6 seats successfully

Total seats left : 4

Amit sorry...!! seats not left

Total seats left : 4

Explanation:

- Here bookSeat() is a **synchronized method**, so **only one thread** can book seats at a time on the same BookMovieSeat object (bts).

- This prevents both threads (*Deepak* and *Amit*) from **booking seats simultaneously**, which could have caused *overbooking* (race condition).
- Because of **synchronization**:
 - When *Deepak* thread starts booking seats, *Amit* must **wait** until *Deepak* finishes execution of `bookSeat()`.
 - After *Deepak* **releases the lock**, *Amit* will execute `bookSeat()`, and check the updated `total_seats`.
- This ensures **correct seat count**, prevents **inconsistent data**, and avoids **lost updates**.

F. Java Thread Control

- Core Java provides complete control over multithreaded program.
- You can develop a multithreaded program which can be suspended, resumed, or stopped completely based on your requirements. There are various static methods which you can use on thread objects to control their behavior.
- **Methods for Controlling Java Thread**
 - a) **public void suspend():** This method puts a thread in the suspended state and can be resumed using `resume()` method.
 - b) **public void stop():** This method stops a thread completely.
 - c) **public void resume():** This method resumes a thread, which was suspended using `suspend()` method.
 - d) **public void wait():** Causes the current thread to wait until another thread invokes the `notify()`.
 - e) **public void notify():** Wakes up a single thread that is waiting on this object's monitor.
- The functions of Suspend, Resume and Stop a thread is performed using **Boolean-type flags** in a **multithreading program**. These flags are used to **store the current status** of the thread.
 - a) If the **suspend** flag is set to **true** then `run()` will **suspend** the **execution** of the currently running thread.
 - b) If the **resume** flag is set to **true** then `run()` will **resume** the **execution** of the suspended thread.
 - c) If the **stop** flag is set to **true** then a thread will get **terminated**.
- Be aware that the latest versions of Java has deprecated the usage of `suspend()`, `resume()`, and `stop()` methods and so you need to use available alternatives.

```
class MyThread implements Runnable{
    Thread t;
    boolean suspendFlag;
    MyThread(String name){
        t = new Thread(this, name);
        System.out.println("Child thread: " + t);
        suspendFlag = false;
        t.start();
    }
    @Override
    public void run() {
        try{
            for(int i = 1; i <= 5; i++){
                System.out.println(t.getName() + ": " + i);
                Thread.sleep(500);
                synchronized(this) {
                    while(suspendFlag)
                        wait();
                }
            }
        }
        catch (InterruptedException e){ System.out.println(t.getName() + " is interrupted!"); }
        System.out.println(t.getName() + " is terminated");
    }
}
```

```

    }
    public synchronized void mySuspend(){    suspendFlag = true;    }
    public synchronized void myResume(){
        suspendFlag = false;
        notify();
    }
}
public class Driver1 {
    public static void main(String[] args){
        MyThread thread1 = new MyThread("First Thread");
        MyThread thread2 = new MyThread("Second Thread");
        try {
            System.out.println("Main thread is waiting...");
            Thread.sleep(1000);
            thread1.mySuspend();
            System.out.println("---Suspending thread 1---");
            Thread.sleep(1000);
            thread1.myResume();
            System.out.println("---Resuming thread 1---");
            Thread.sleep(1000);
            thread1.t.join();                thread2.t.join();
        }
        catch (InterruptedException e){
            System.out.println("Main thread is interrupted!");
        }
        System.out.println("Main thread terminated");
    }
}

```

Output:

```

Child thread: Thread[#20,First Thread,5,main]
Child thread: Thread[#21,Second Thread,5,main]
Main thread is waiting...
Second Thread: 1
First Thread: 1
Second Thread: 2
First Thread: 2
---Suspending thread 1---
Second Thread: 3
Second Thread: 4
---Resuming thread 1---
First Thread: 3
Second Thread: 5
First Thread: 4
Second Thread is terminated
First Thread: 5
First Thread is terminated
Main thread terminated

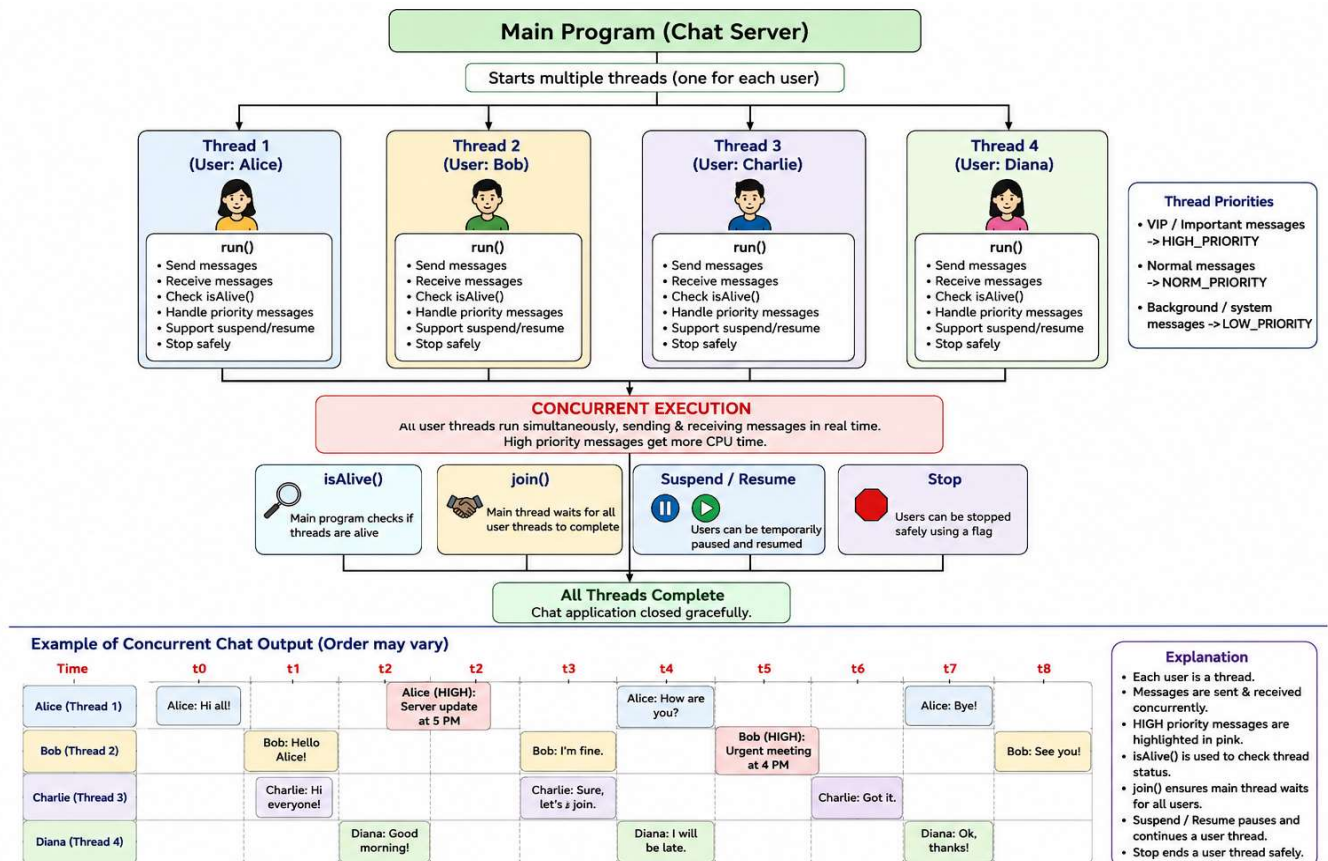
```

- In short

Operation	Java Approach
Suspend	wait() + control flag
Resume	notify() / notifyAll()
Stop	Control flag + interrupt()
Check Status	isAlive()
Wait for Completion	join()

G. In short Problem statement

- Develop a **multithreaded Java application** that simulates a real-time **basic chat system**, where each user is represented by an independent thread. Each user thread should be able to **send and receive messages concurrently**, demonstrating how multiple users can interact at the same time. The application should use **thread priorities** to give greater importance to high-priority or urgent messages, while normal messages continue to be processed by other threads.
- The application should also demonstrate important **thread management and synchronization techniques**. The `isAlive()` method should be used to check whether user threads are currently active, while `join()` should ensure that the main program waits for all user threads to complete before terminating. The system should demonstrate **safe suspension, resumption, and stopping of threads** using control flags and interruption, rather than unsafe deprecated thread-control methods. Overall, the application should help demonstrate the **creation, concurrent execution, monitoring, synchronization, priority handling, and safe termination of multiple Java threads** in a real-time scenario.



7. Implementation:

Program for implementation of Chat Application

```
class ChatUser extends Thread {
    private String userName;
    private boolean suspended = false;
    private boolean running = true;
    public ChatUser(String userName) {
        this.userName = userName;
    }
    public synchronized void suspendUser() {
        suspended = true;
        System.out.println(userName + " suspended.");
    }
    public synchronized void resumeUser() {
        suspended = false;
    }
}
```

```

        notify();
        System.out.println(userName + " resumed.");
    }
    public void stopUser() {
        running = false;
        System.out.println(userName + " stopped.");
    }
    @Override
    public void run() {
        int count = 1;
        try {
            while (running && count <= 5) {
                synchronized (this) {
                    while (suspended) {
                        wait();
                    }
                }
                System.out.println(userName + " sends message : Hello " + count);
                Thread.sleep(1000);
                count++;
            }
        } catch (InterruptedException e) {
            System.out.println(userName + " interrupted.");
        }
        System.out.println(userName + " exited chat.");
    }
}

public class ChatSystem {
    public static void main(String[] args) throws InterruptedException {
        ChatUser user1 = new ChatUser("Alice");
        ChatUser user2 = new ChatUser("Bob");
        ChatUser user3 = new ChatUser("Admin");
        // Thread Priorities
        user1.setPriority(Thread.NORM_PRIORITY);
        user2.setPriority(Thread.MIN_PRIORITY);
        user3.setPriority(Thread.MAX_PRIORITY);
        System.out.println("Starting Chat System...\n");
        user1.start();
        user2.start();
        user3.start();
        // Check status using isAlive()
        System.out.println("Alice Alive : " + user1.isAlive());
        System.out.println("Bob Alive : " + user2.isAlive());
        System.out.println("Admin Alive : " + user3.isAlive());
        Thread.sleep(3000);
        // Suspend Bob
        user2.suspendUser();
        Thread.sleep(3000);
        // Resume Bob
        user2.resumeUser();
        Thread.sleep(2000);
        // Stop Alice
        user1.stopUser();
        // Synchronization using join()
        user1.join();
        user2.join();
        user3.join();
        System.out.println("\nAll users have left the chat.");
        System.out.println("Chat System Closed.");
    }
}

```

Output:

```

Admin sends message : Hello 2
Alice sends message : Hello 2
Bob sends message : Hello 2
Admin sends message : Hello 3
Alice sends message : Hello 3
Bob sends message : Hello 3
Bob suspended.

```

```

Alice sends message : Hello 4
Admin sends message : Hello 4
Admin sends message : Hello 5
Alice sends message : Hello 5
Admin exited chat.
Alice exited chat.
Bob resumed.
Bob sends message : Hello 4
Bob sends message : Hello 5
Alice stopped.
Bob exited chat.
All users have left the chat.
Chat System Closed.

```

8. Conclusion:

This problem demonstrates how Java multithreading can be applied to build a real-time chat system where multiple users communicate concurrently. It provides practical understanding of thread creation, priorities, status checking, synchronization, and safe thread control, helping students understand the complete thread lifecycle in a real-world application.

9. Review Questions & Exercises:

a. Fill in the Blanks

Q. No.	Question	BT Level	CO	PO & PSO
ii	Multiple threads executing during the same time period demonstrate _____ execution.	BT ₁	CO ₂	PO ₁ , PSO ₁
iii	Safe thread termination is also called _____ thread termination because the thread cooperates with the stop request.	BT ₁	CO ₂	PO ₁ , PSO ₁

b. Answer the following:

Q. No.	Question	BT Level	CO	PO & PSO
i	Explain how multiple user threads can send and receive messages concurrently in a chat application.	BT ₂	CO ₂	PO ₁ , 2, PSO ₁
ii	How do wait() and notify() work?	BT ₂	CO ₂	PO ₁ , 2, PSO ₁

c. Programming Assignments:

Q. No.	Question	BT Level	CO	PO & PSO
i	Online Examination System: Develop a multithreaded Java application that simulates an Online Examination System where each student is represented by a separate thread attempting the exam simultaneously. Use the <code>isAlive()</code> method to monitor active student threads and the <code>join()</code> method to ensure the examiner waits until all students have submitted before ending the examination and declaring the result.	BT ₃	CO ₂	PO _{1,3,5} , PSO ₁

10. References:

- E Balaguruswamy, (2023). Programming with Java: A Primer. 7th. India: McGraw Hill Education
- Herbert Schildt, (2021). Java: The complete reference, 13th edition. McGraw-Hill Education.

Performance Indicators	
Process related (6 Marks)	
1	Logic formation (3)
2	Debugging ability (2)
3	Follow ethical practices (1)
Product related (4 Marks)	
4	Expected output (1)
5	Timely Submission (2)
6	Answer to sample questions (1)

Marks Obtained			Dated signature of Teacher
Process Related (6)	Product Related (4)	Total (10)	